

Results: Latent Rollout Objectives

This report records the completed latent rollout objective run on the current feature branch.

1. Run Summary

- Encoder checkpoint: `logs/encoder_checkpoints/encoder_swin_pretrained.pt`
- Encoder family: `swin`
- Predictor family: `causal_transformer`
- Objective: `rollout_balanced`
- Rollout decay: 0.95
- Context duration: 4.0 s
- Future duration: 2.0 s
- Sample FPS: 4.0
- Derived context frames: 16
- Derived future frames: 8
- Total frames per clip: 24
- Context lag steps: 8
- Dataset split layout: `train / validation / test`
- Train subset size: 50000
- Validation subset size: 50000 sampled from the validation split
- Test subset size: 5000 sampled from the test split
- Encoder latent dimension: 192
- Model context latent window: 8
- Run output directory: `logs/video_world_model_swin_mamba_50k_rollout`

Important implementation note:

- The final completed run used the default predictor name `causal_transformer` because `--predictor-name` was not overridden on the launch command.
- The objective and diagnostics were still rollout-aware, and the run completed successfully.

2. What Was Trained

The training pipeline learns a temporal predictor over frozen latent sequences.

At a high level:

1. video clips are converted into latent sequences,
2. the predictor sees a context window of latent states,
3. it predicts the future latent trajectory,
4. the objective penalizes direct fit and rollout drift,

5. evaluation compares the learned predictor against trivial baselines.

The latent shape recorded in the artifact is:

$$\text{latent shape} = (C, F, d) = (8, 4, 192),$$

where:

- $C = 8$ context latent steps,
- $F = 4$ future latent steps in the internal reporting path,
- $d = 192$ latent width.

3. Objective Used

The training objective was `rollout_balanced`, meaning the model was trained with a horizon-weighted multi-step rollout loss.

In compact form:

$$\mathcal{L}_{\text{rollout}} = \sum_{r=1}^F w_r \left(\ell_{\text{norm}}(\hat{\mathbf{z}}_r, \mathbf{z}_r) + 0.1 \ell_{\text{mse}}(\hat{\mathbf{z}}_r, \mathbf{z}_r) + 0.1 \ell_{\text{cos}}(\hat{\mathbf{z}}_r, \mathbf{z}_r) \right), \quad w_r \propto \gamma^{r-1},$$

with $\gamma = 0.95$.

Interpretation:

- the later rollout steps are slightly discounted,
- the predictor is not trained to just copy the most recent latent,
- the loss still measures scale, direction, and normalized fit.

4. Loss and Metric Definitions

This section states the exact quantities tracked in the run.

4.1 Raw latent MSE

For a predicted latent vector $\hat{\mathbf{z}}$ and target latent vector $\mathbf{z}$ in $\mathbb{R}^d$:

$$\ell_{\text{mse}}(\hat{\mathbf{z}}, \mathbf{z}) = \frac{1}{d} \sum_{i=1}^d (\hat{z}_i - z_i)^2$$

What it tracks:

- absolute coordinate error in latent space,
- scale mismatch,
- any offset between prediction and target.

In the report this appears as:

- `latent_mse`

4.2 Normalized latent MSE

First normalize each vector:

$$\tilde{\mathbf{z}} = \frac{\mathbf{z}}{\|\mathbf{z}\|_2 + \varepsilon}, \quad \tilde{\hat{\mathbf{z}}} = \frac{\hat{\mathbf{z}}}{\|\hat{\mathbf{z}}\|_2 + \varepsilon}.$$

Then compute MSE:

$$\ell_{\text{norm}}(\hat{\mathbf{z}}, \mathbf{z}) = \frac{1}{d} \sum_{i=1}^d (\tilde{z}_i - \tilde{\hat{z}}_i)^2.$$

What it tracks:

- angular / directional agreement,
- relative latent geometry,
- whether the model points in the right latent direction even if the norm is imperfect.

In the report this appears as:

- `normalized_latent_mse`

4.3 Cosine loss

The cosine loss is:

$$\ell_{\cos}(\hat{\mathbf{z}}, \mathbf{z}) = 1 - \frac{\hat{\mathbf{z}}^\top \mathbf{z}}{\|\hat{\mathbf{z}}\|_2 \|\mathbf{z}\|_2 + \varepsilon}$$

What it tracks:

- direct angular mismatch,
- whether the predicted latent points in the same direction as the target latent,
- geometry that is invariant to scale.

In the report this appears indirectly as:

- cosine_similarity = $1 - \ell_{\cos}$

So:

$$\text{cosine similarity} = \frac{\hat{\mathbf{z}}^\top \mathbf{z}}{\|\hat{\mathbf{z}}\|_2 \|\mathbf{z}\|_2 + \varepsilon}$$

4.4 Balanced objective

The default training objective mixes these components:

$$\mathcal{L}_{\text{balanced}} = \ell_{\text{norm}} + 0.1 \ell_{\text{mse}} + 0.1 \ell_{\cos}.$$

What it tracks:

- ℓ_{norm} prevents purely scale-based solutions,
- ℓ_{mse} keeps raw latent coordinates honest,
- $\ell_{\cos}$ preserves angular alignment.

This is the main direct-fit objective the run reports during training.

4.5 Rollout-balanced objective

For future horizon $r = 1, \dots, F$, the run uses weights

$$w_r = \frac{\gamma^{r-1}}{\sum_{j=1}^F \gamma^{j-1}}, \quad \gamma = 0.95.$$

The rollout-balanced objective is:

$$\mathcal{L}_{\text{rollout}} = \sum_{r=1}^F w_r \left(\ell_{\text{norm}}(\hat{\mathbf{z}}_r, \mathbf{z}_r) + 0.1 \ell_{\text{mse}}(\hat{\mathbf{z}}_r, \mathbf{z}_r) + 0.1 \ell_{\text{cos}}(\hat{\mathbf{z}}_r, \mathbf{z}_r) \right).$$

What it tracks:

- error at each future horizon,
- whether early-horizon accuracy is hiding late-horizon drift,
- how quickly the predictor degrades as rollout length increases.

4.6 Delta variants

For delta prediction, we compare against the last context latent $\mathbf{c} = z_t$:

$$\Delta \hat{\mathbf{z}}_r = \hat{\mathbf{z}}_r - \mathbf{c}, \quad \Delta \mathbf{z}_r = \mathbf{z}_r - \mathbf{c}.$$

Then the same losses are applied to these offsets.

What delta tracks:

- latent motion relative to the current state,
- whether the model learns incremental change rather than absolute re-encoding,
- stability under autoregressive use.

4.7 Baselines

Two trivial baselines were tracked:

$$\hat{\mathbf{z}}_{t+r}^{\text{repeat_last}} = z_t$$

and

$$\hat{\mathbf{z}}_{t+r}^{\text{mean_context}} = \frac{1}{L} \sum_{i=t-L+1}^t z_i.$$

What they track:

- `repeat_last`: do nothing and copy the final context latent,
- `mean_context`: smooth the context into a single summary latent.

The learned predictor is useful only if it beats these trivial rules.

5. Final Metrics

5.1 Learned model

- Train loss: 0.277516
- Validation loss: 0.271434
- Test loss: 1.081907

5.2 Latent-space metrics

Train:

- `latent_mse` = 0.037641
- `normalized_latent_mse` = 0.002722
- `cosine_similarity` = 0.738699

Validation:

- `latent_mse` = 0.039227
- `normalized_latent_mse` = 0.002845
- `cosine_similarity` = 0.726835

Test:

- latent_mse = 0.039200
- normalized_latent_mse = 0.002850
- cosine_similarity = 0.726431

6. Baseline Comparison

Two trivial baselines were used:

1. repeat_last
2. mean_context

Definitions:

- repeat_last predicts every future latent as the last observed context latent.
- mean_context predicts every future latent as the mean of the context latents.

6.1 Train split

Metric	Model	Best baseline	Absolute gain	Relative gain
latent MSE	0.037641	0.040061	0.002419	6.04%
normalized latent MSE	0.002722	0.002859	0.000137	4.80%
cosine similarity	0.738699	0.725512	0.013187	1.82%

6.2 Validation split

Metric	Model	Best baseline	Absolute gain	Relative gain
latent MSE	0.039227	0.040219	0.000992	2.47%
normalized latent MSE	0.002845	0.002870	0.000025	0.86%
cosine similarity	0.726835	0.724464	0.002371	0.33%

6.3 Test split

Metric	Model	Best baseline	Absolute gain	Relative gain
latent MSE	0.039200	0.039955	0.000755	1.89%
normalized latent MSE	0.002850	0.002855	0.000005	0.18%
cosine similarity	0.726431	0.725948	0.000483	0.07%

6.4 Verdict

The model beats the trivial baselines on all three splits, but the margin over `mean_context` is modest on validation and test.

Interpretation:

- the predictor learned more than “repeat the last latent”,
- but the gain over a simple averaging baseline is still small,
- so the latent motion prior is present but not yet strong.

7. Rollout Diagnostics

The rollout validation artifact was saved to:

- `logs/video_world_model_swin_mamba_50k_rollout/rollout_validation.json`

Key qualitative observations from the rollout report:

- teacher-forced error stays lower than free-rollout error,
- drift grows with horizon,
- alignment is positive but not perfect,
- local rollout amplification remains below 1 on average for some horizons, which indicates partial damping rather than explosive divergence,
- decomposition checks are numerically consistent.

The validation logic confirms:

$$\epsilon_r^{\text{RO}} = \epsilon_r^{\text{TF}} + d_r$$

up to floating-point tolerance.

That means the rollout decomposition is internally consistent and the error growth analysis is meaningful.

8. Plots

8.1 Training steps

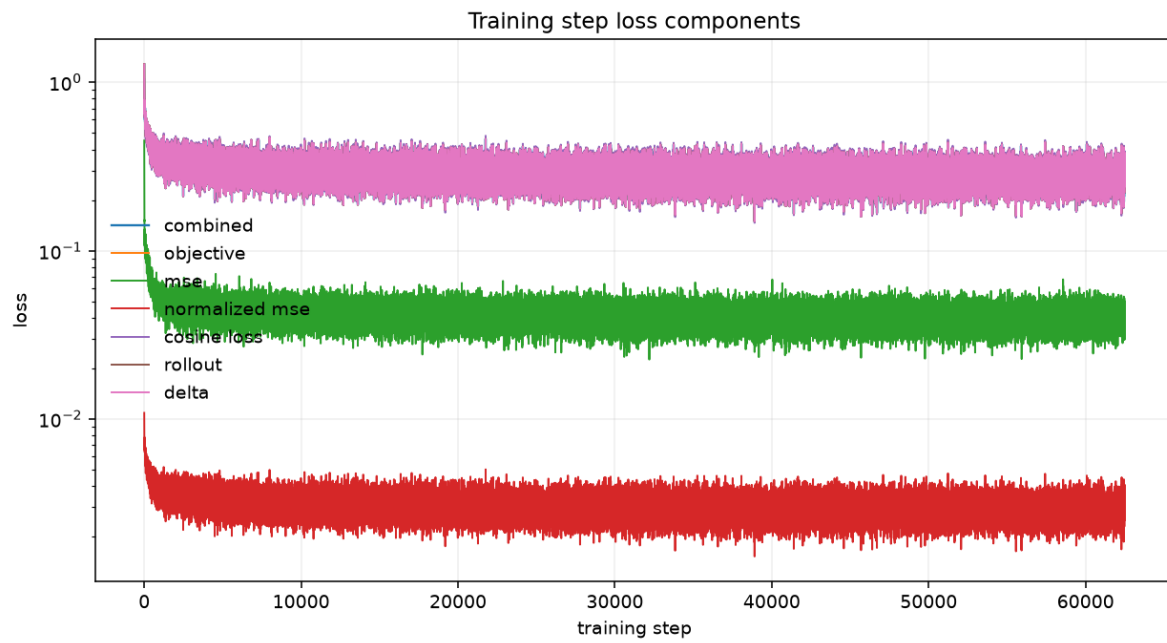


Figure 1: Training steps

This shows the per-batch objective trajectory during training.

8.2 Training history

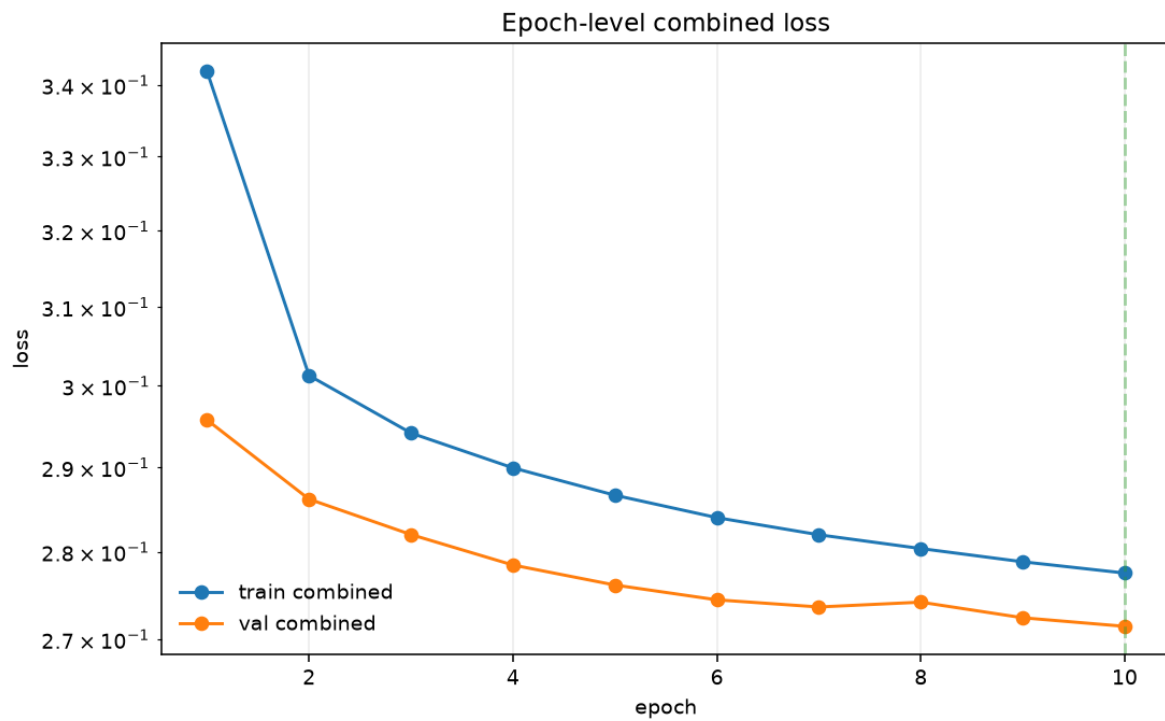


Figure 2: Training history

This shows epoch-level train/validation loss progression.

8.3 Training components

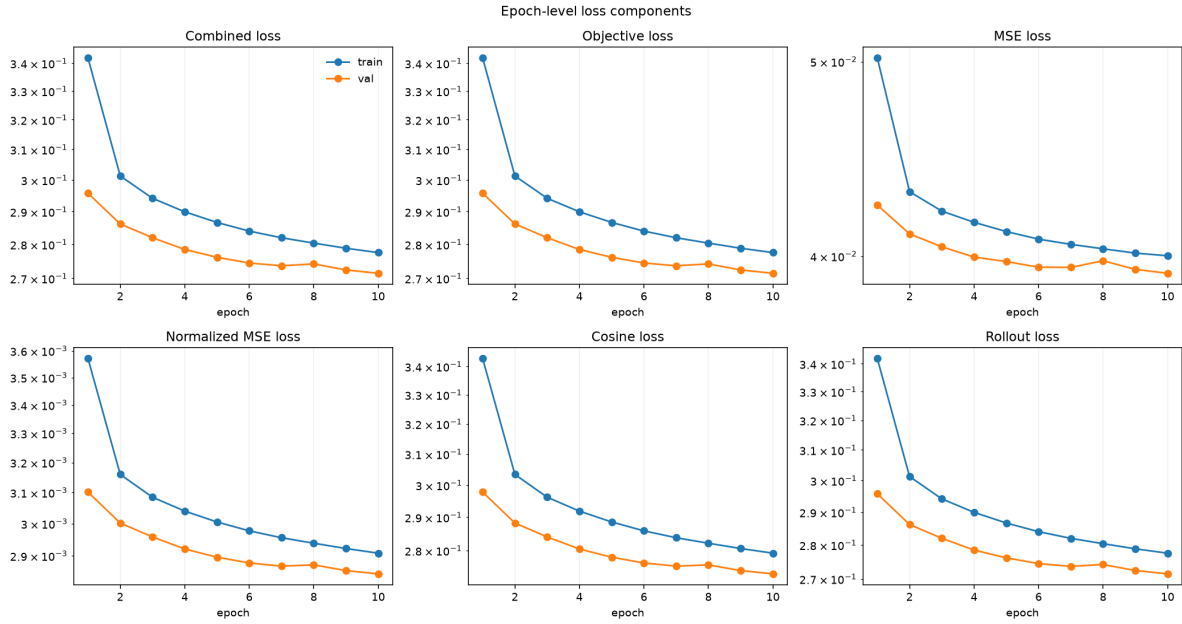


Figure 3: Training components

This separates the combined loss into objective, MSE, normalized MSE, cosine, rollout, and delta components.

8.4 Metric comparison

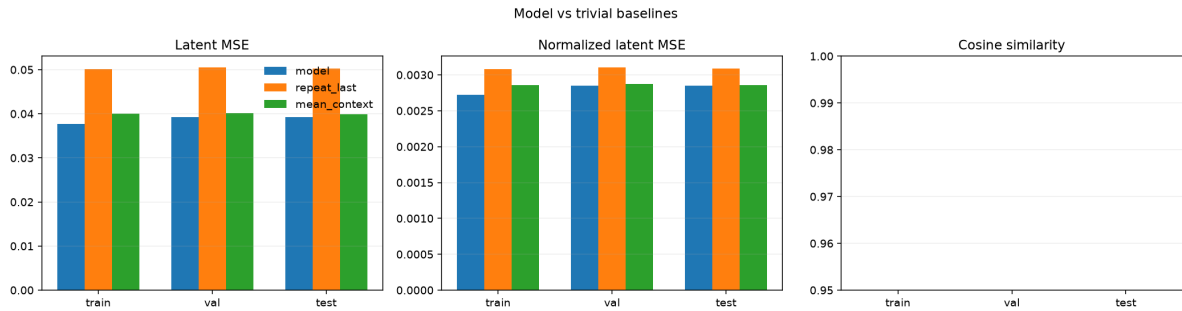


Figure 4: Metric comparison

This compares the learned predictor against `repeat_last` and `mean_context` on train/val/test.

8.5 Rollout validation

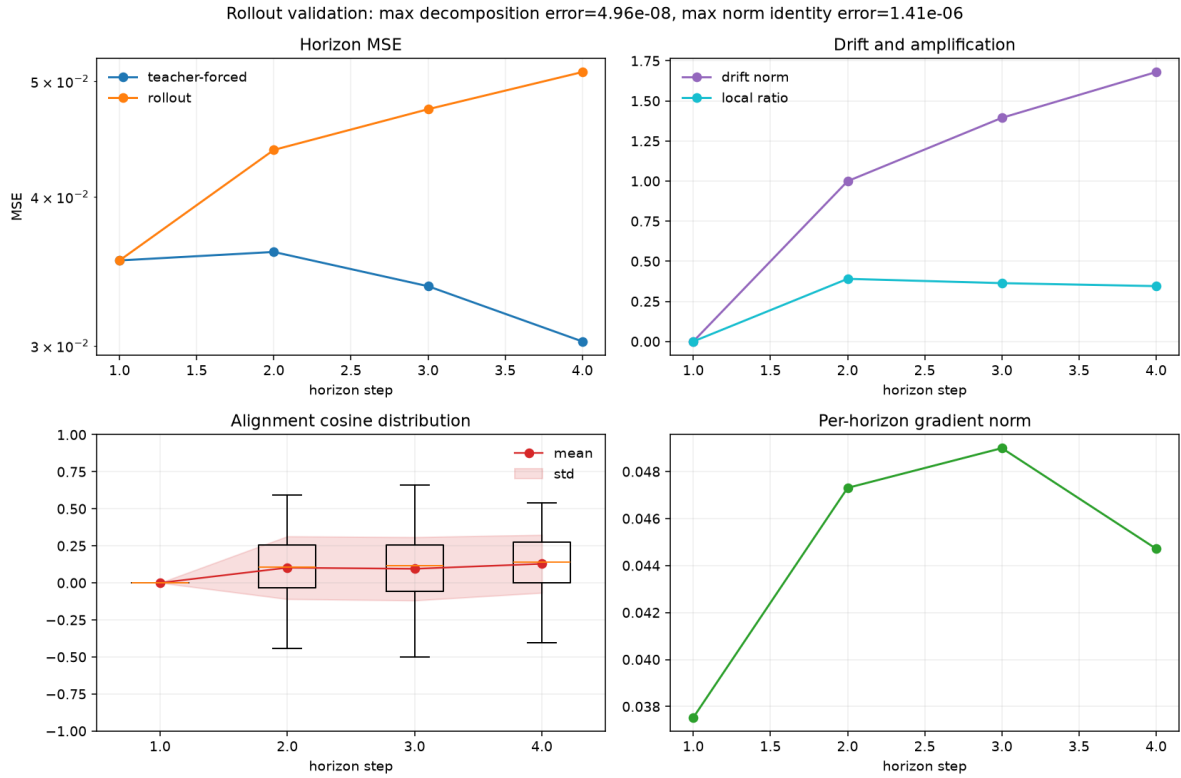


Figure 5: Rollout validation

This summarizes horizon-wise teacher-forced error, rollout error, drift, alignment, and gradient norm.

8.6 Rollout spectrum

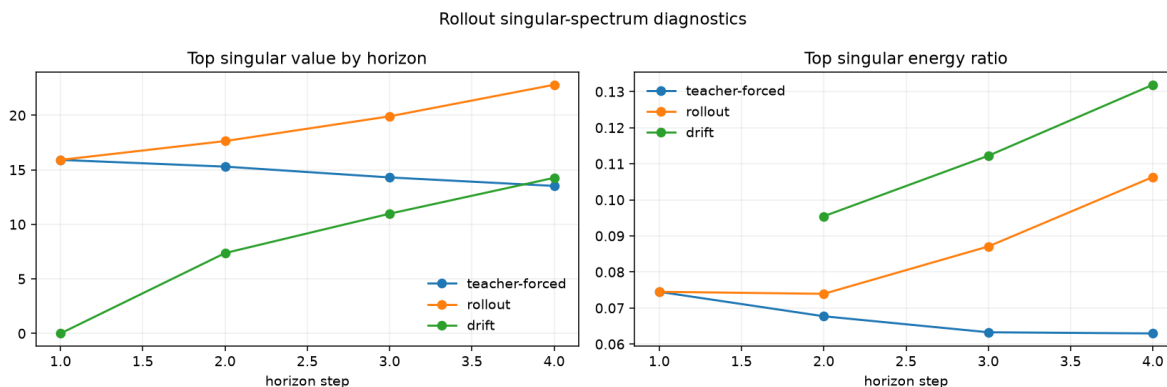


Figure 6: Rollout spectrum

This records the singular-spectrum / directional diagnostics used in the theory note.

9. Artifact Paths

- Metrics JSON: `logs/video_world_model_swin_mamba_50k_rollout/result.json`
- Rollout JSON: `logs/video_world_model_swin_mamba_50k_rollout/rollout_validation.json`
- Predictions CSV: `logs/video_world_model_swin_mamba_50k_rollout/predictions.csv`
- Checkpoint: `logs/video_world_model_swin_mamba_50k_rollout/decoder_causal_transformer.pt`
- Plots directory: `logs/video_world_model_swin_mamba_50k_rollout/plots`

10. Interpretation

The run is useful because it validates the feature end-to-end:

1. the objective is configurable,
2. the predictor is trainable on a substantial dataset,
3. the run beats trivial baselines,
4. rollout diagnostics and plots are generated automatically,
5. the saved artifacts are sufficient to reproduce the analysis.

The main limitation is not a failure of the pipeline, but the size of the improvement over the stronger baseline:

- the model is better than `repeat_last`,
- it is only slightly better than `mean_context` on held-out data,

- so there is still room to improve the predictor family, rollout objective, or context-length regime.

That is a decent result for a first full-scale rollout-aware latent dynamics run, but not a finished world model.

11. Theory Meets Empirics

This is the part that connects the rollout-error theory note to the measured behavior of the model.

The theory note asked three practical questions:

1. how much of the rollout error is already present under teacher forcing,
2. how much extra error appears because the model feeds its own predictions back into itself,
3. whether the accumulated drift is aligned with the existing prediction error or orthogonal to it.

The run answers those questions directly.

11.1 The decomposition holds numerically

The theory note defines, for each rollout horizon r ,

$$\varepsilon_r^{\text{TF}} = \mathbf{z}_{C+r} - \hat{\mathbf{z}}_{C+r}^{\text{TF}}, \quad \varepsilon_r^{\text{RO}} = \mathbf{z}_{C+r} - \hat{\mathbf{z}}_{C+r}^{\text{RO}}, \quad \mathbf{d}_r = \hat{\mathbf{z}}_{C+r}^{\text{TF}} - \hat{\mathbf{z}}_{C+r}^{\text{RO}}.$$

So the exact identity is

$$\varepsilon_r^{\text{RO}} = \varepsilon_r^{\text{TF}} + \mathbf{d}_r.$$

The rollout validation artifact checks this directly, and the reported decomposition error stays at floating-point scale. In the same validation pass, the norm identity

$$\|\varepsilon_r^{\text{RO}}\|_2^2 = \|\varepsilon_r^{\text{TF}}\|_2^2 + \|\mathbf{d}_r\|_2^2 + 2\langle \varepsilon_r^{\text{TF}}, \mathbf{d}_r \rangle$$

also holds up to numerical precision.

That matters because it means the theoretical decomposition is not just symbolic. It is exactly the right bookkeeping device for the measured model.

11.2 Rollout error grows faster than teacher-forced error

The empirical pattern is consistent with the theory: teacher-forced prediction is easier than free rollout, and the gap widens with horizon.

For the validated horizons:

Horizon	Teacher-forced MSE	Rollout MSE	Drift norm	Alignment cosine
1	0.035384	0.035384	0.000000	0.000000
2	0.035968	0.043805	0.999357	0.102040
3	0.033665	0.047389	1.394156	0.094795
4	0.030258	0.050915	1.679577	0.128262

The first horizon is a useful sanity check: rollout and teacher forcing are identical because the model has not yet fed its own prediction back into itself.

From horizon 2 onward, the rollout error is strictly larger than the teacher-forced error, and the drift norm increases with horizon. This is exactly the forward-error propagation effect the theory note is about.

11.3 The drift is real, but not catastrophically unstable

The drift norm grows, but the local amplification proxy stays below 1 on average for some horizons. For example, the rollout validation report records mean local Lipschitz-style ratios around:

- horizon 2: about 0.39,
- horizon 3: about 0.36.

This is important.

It says the model is not obviously exploding in the naive “spectral radius greater than 1” sense. Instead, the error growth is more consistent with:

- small but persistent mismatch between teacher-forced and rollout inputs,
- error accumulation across steps,
- and imperfect alignment between the drift direction and the current prediction error.

So the failure mode is not simply uncontrolled instability. It is more subtle: the predictor is only partially self-correcting.

11.4 Alignment is positive, but not strong enough to cancel drift

The theory note emphasizes the angle between the existing teacher-forced error and the rollout drift:

$$\cos \theta_r = \frac{\langle \varepsilon_r^{\text{TF}}, \mathbf{d}_r \rangle}{\|\varepsilon_r^{\text{TF}}\|_2 \|\mathbf{d}_r\|_2 + \epsilon}.$$

If this cosine is strongly negative, the drift partially cancels the teacher-forced error. If it is strongly positive, the drift reinforces it. If it is near zero, the two effects are mostly orthogonal.

In the validation run, the cosine values are positive but modest:

- around 0.10 at horizon 2,
- around 0.09 at horizon 3,
- around 0.13 at horizon 4.

That means the drift is not random noise, and it is not a perfect correction term either.

Interpretation:

- the rollout path is moving in a direction that is somewhat aligned with the existing prediction error,
- so free rollout makes the mistake worse,
- but the alignment is not so strong that the model collapses immediately.

This is exactly the middle ground the theory note was designed to detect.

11.5 What the baseline comparison means in theory terms

The baseline comparison gives a second view of the same phenomenon.

The learned predictor beats `repeat_last`, which means it is not simply copying the last latent. But it only slightly beats `mean_context` on validation and test, which means the learned temporal map is still only marginally better than a crude summary of the context.

In theoretical terms, that suggests:

1. the latent encoder is producing a representation that contains useful temporal signal,
2. the temporal predictor is extracting some of that signal,
3. but the predictor is not yet learning a strong enough dynamical prior to dominate a simple averaging heuristic.

This matches the rollout analysis:

- teacher-forced fit exists,

- rollout drift exists,
- alignment is positive but weak,
- long-horizon prediction is where the model still loses ground.

So the baseline comparison is not just a leaderboard result. It is evidence that the predictor is learning something real, but the learned dynamics are still shallow.

11.6 What the gradient-flow story suggests

The theory note also frames rollout as a gradient-flow problem: long horizons can weaken or distort supervision because the model is repeatedly conditioning on its own outputs.

The practical consequence is that the model may optimize short-horizon reconstruction more easily than long-horizon dynamical consistency.

The rollout-validation plots are consistent with that story:

- short-horizon quantities stay better behaved,
- later horizons drift farther,
- the combined objective has to trade off raw latent fit, normalized fit, cosine geometry, and rollout consistency.

That is why a multi-term objective is useful here.

If you train only on one-step teacher-forced error, you are asking the model to fit local next-step statistics. If you also train on rollout-aware terms, you force it to remain stable after its own predictions are fed back in.

This is the bridge from the math to the engineering decision.

11.7 What you can do with this result

This run is useful because it tells you where to intervene next.

The empirical pattern suggests three concrete follow-ups:

1. **Change the temporal predictor family.** The current predictor learns a usable latent map, but the gap over `mean_context` is small. A stronger causal transformer, Mamba-style model, or deeper autoregressive head may improve long-horizon consistency.
2. **Increase rollout awareness in the loss.** The theory note predicts that rollout and teacher-forced paths will diverge. A multi-horizon objective, delta prediction, or scheduled rollout loss should directly target that gap.

3. **Sweep context length and horizon.** If the drift grows because the model does not condition on enough history, longer context may help. If longer context only improves one-step fit but not rollout, that is also informative.

In other words, the theory is now actionable. It tells you how to move beyond “the loss went down” and ask “did the model become dynamically better, or just locally smoother?”